

BUỔI 1

Môi trường lập trình Python, biến, kiểu dữ liệu và toán tử



The Day I Realized Programming Is Just Like
Making Coffee
by Amit Kumar | Mar, 2026 | Medium



Buổi 1

210 phút ·
3,5 giờ



Lý thuyết
+ thực hành



Đối tượng:

Người mới bắt đầu
học lập trình Python



Học hôm nay – Kiến tạo tương lai



Small
Steps
Big
Programmers



I. Mục tiêu bài học

Sau buổi học, học viên có thể:

- 1  Trình bày được **Python** là gì và một số ứng dụng phổ biến.
- 2  Cài đặt, cấu hình **môi trường Python và IDE**.
- 3  Viết, lưu và chạy chương trình Python bằng **IDE** hoặc **Terminal**.
- 4  Khai báo biến, nhận biết và sử dụng các kiểu dữ liệu **int, float, str, bool, None**.
- 5  Sử dụng quy tắc đặt tên biến theo **PEP 8**.
- 6  Vận dụng **toán tử số học, gán, so sánh, logic và membership**.
- 7  Hoàn thành các **bài tập lập trình cơ bản** có sử dụng biến, kiểu dữ liệu và toán tử.



1.1

Python là gì?

Python là một ngôn ngữ lập trình bậc cao, có cú pháp tương đối dễ đọc, được sử dụng trong nhiều lĩnh vực như:



Phát triển ứng dụng và website

Xây dựng phần mềm, website, ứng dụng web



Phân tích dữ liệu

Xử lý và phân tích dữ liệu hiệu quả



Trí tuệ nhân tạo và học máy

Phát triển các mô hình AI, Machine Learning



Tự động hóa các công việc

Tự động hóa các tác vụ lặp đi lặp lại



Giáo dục và nghiên cứu khoa học

Ngôn ngữ phổ biến trong giảng dạy và nghiên cứu



*One language
Many possibilities*



"Python giúp biến ý tưởng của bạn thành hiện thực!"

Simple
Powerful
Versatile

> Ví dụ chương trình Python đầu tiên

```
hello.py
1 print("Hello, Python!")
```

▶ Kết quả:

Hello, Python!



Giải thích:

- `print()` là hàm dùng để hiển thị dữ liệu ra màn hình.
- `"Hello, Python!"` là một chuỗi ký tự.
- Nội dung bên trong dấu ngoặc được truyền vào hàm `print()`.



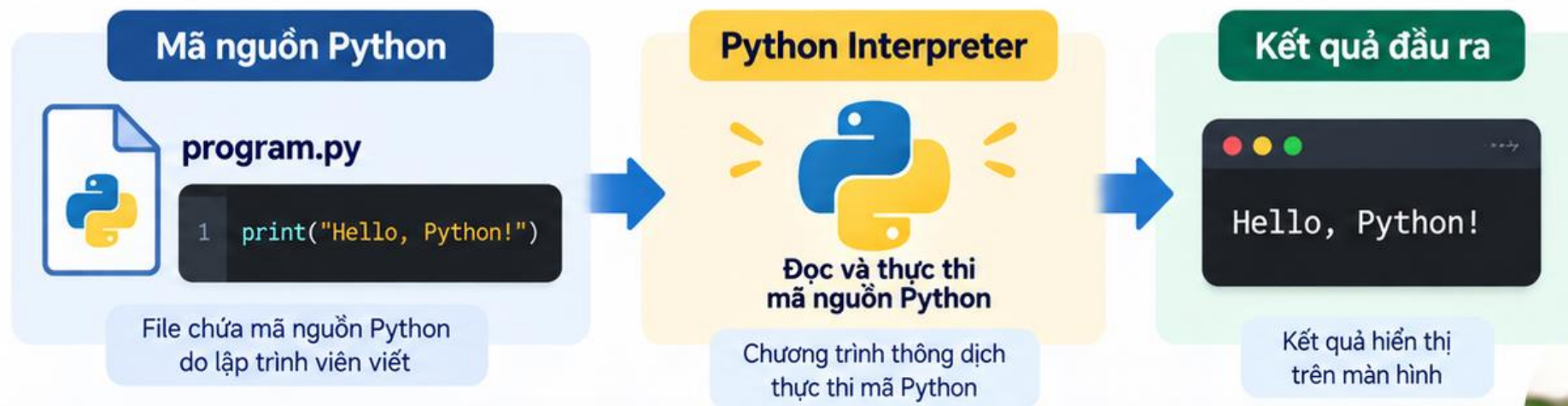
*A
Brighter
Future
with Python*



1.2

Python hoạt động như thế nào?

Code Today
A Better Tomorrow



Các thành phần cần phân biệt:

Thành phần	Vai trò
Python	Ngôn ngữ lập trình
Python Interpreter	Chương trình thực thi mã Python
IDE	Phần mềm hỗ trợ viết và chạy mã
Terminal	Giao diện dòng lệnh để nhập lệnh và chạy chương trình



Cùng một mã nguồn Python có thể chạy trên nhiều nền tảng khác nhau!



Small Steps
Big Programmers



1.3

Một số môi trường lập trình phổ biến

Chọn đúng công cụ để học hiệu quả hơn!



Visual Studio Code (VS Code)

Trình soạn thảo mã nguồn nhẹ, hỗ trợ Python thông qua **extension**. Phù hợp để học lập trình và phát triển dự án.



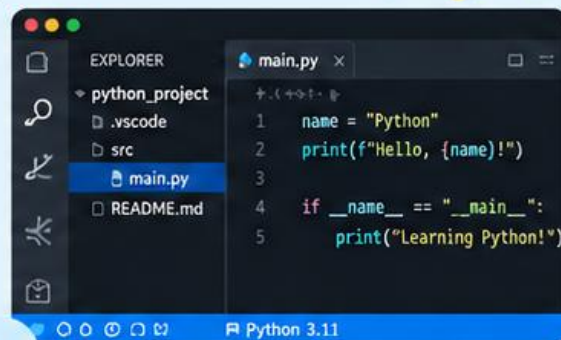
Nhẹ, nhanh



Hỗ trợ extension



Dễ sử dụng



Code with VS Code



PyCharm

IDE chuyên dụng cho Python, tích hợp nhiều công cụ hỗ trợ viết, gỡ lỗi và quản lý dự án.



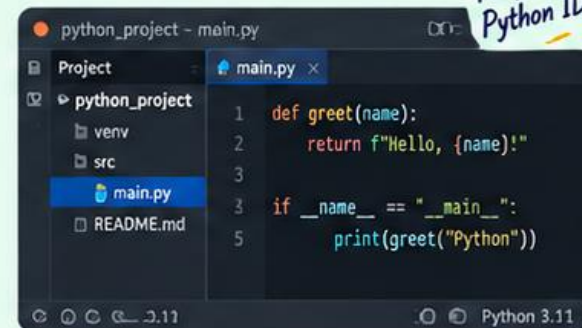
Đầy đủ tính năng



Gỡ lỗi mạnh mẽ



Quản lý dự án tốt



Professional Python IDE



Jupyter Notebook / Google Colab

Cho phép viết và chạy mã theo từng ô, thuận tiện cho giảng dạy, thực nghiệm và phân tích dữ liệu.



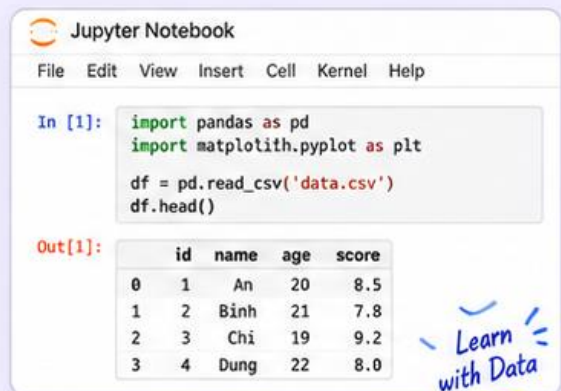
Chạy từng ô



Không cần cài đặt



Phân tích dữ liệu



Learn with Data



Thonny

IDE đơn giản, giao diện dễ tiếp cận, phù hợp với người mới bắt đầu.



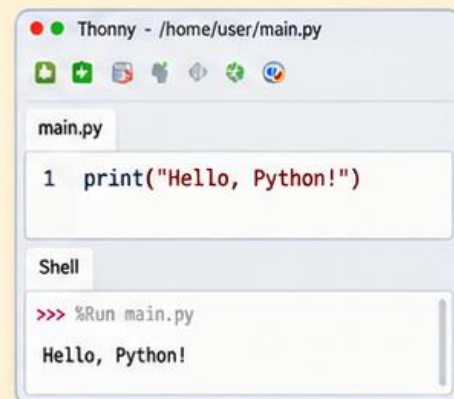
Dễ học



Tối ưu cho người mới



Có sẵn Python



Simple & Friendly



Mỗi công cụ có điểm mạnh riêng. Hãy chọn môi trường phù hợp với mục tiêu của bạn!



Thảo luận nhanh

Python là ngôn ngữ lập trình hay phần mềm lập trình?

1



IDE có phải là Python không?

2



Muốn chạy một chương trình Python thì cần những thành phần nào?

3



Theo em, **Python** có thể được sử dụng để giải quyết những công việc gì?

Cùng nhau suy nghĩ và chia sẻ ý kiến nhé!



Think
Code
Share
Grow

HỌC HỎI
THẢO LUẬN
TIẾN BỘ
CÙNG NHAU
😊



Python – Mở khóa tiềm năng, kiến tạo tương lai!

“Những ý tưởng lớn thường bắt đầu từ những câu hỏi đơn giản!”

Better
Questions
Brighter
Solutions



2.1. Cài đặt Python

Giảng viên hướng dẫn học viên thực hiện:

1 Truy cập trang tải Python

Mở python.org/downloads và tải phiên bản Python 3 phù hợp với hệ điều hành.



<https://www.python.org/downloads>

Luôn tải phiên bản Python 3 mới nhất và phù hợp với hệ điều hành của bạn (Windows, macOS, Linux).

2 Cài đặt Python

Trên Windows, đánh dấu tùy chọn **Add Python to PATH** nếu trình cài đặt cung cấp tùy chọn này, sau đó tiến hành cài đặt.



Lưu ý quan trọng:

Hãy đánh dấu "Add Python to PATH" để có thể chạy Python từ Terminal / Command Prompt.

3 Kiểm tra phiên bản Python

Mở Terminal hoặc Command Prompt và nhập:

```
Windows (Command Prompt)
python --version
Python 3.12.2
```

Nếu không hoạt động trên Windows, thử:

```
py --version
Python 3.12.2
```

Trên macOS/Linux, thường sử dụng:

```
python3 --version
Python 3.12.2
```



Kết quả hiển thị phiên bản Python tương tự như hình là cài đặt thành công.

4 Kiểm tra Python Interpreter

Chạy lệnh:

```
python
```

Nếu vào được chế độ tương tác Python, nhập:

```
>>> print("Python is ready!")
Python is ready!
>>>
```

Thoát bằng **exit()** hoặc tổ hợp phím phù hợp với Terminal.



Bạn đã sẵn sàng để viết và chạy những chương trình Python đầu tiên!

Let's
get started!



Small Setup
Big Possibilities



"A journey of a thousand lines
begins with a single setup."

Good
Ideas
Start
Here ☺



2.2

Cấu hình VS Code

Good Tools
Better Code

1 Cài đặt Visual Studio Code

Truy cập <https://code.visualstudio.com> và tải về cài đặt.



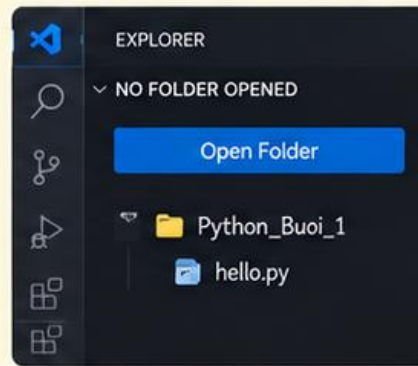
2 Cài extension Python

Mở VS Code và cài i extension Python của Microsoft.



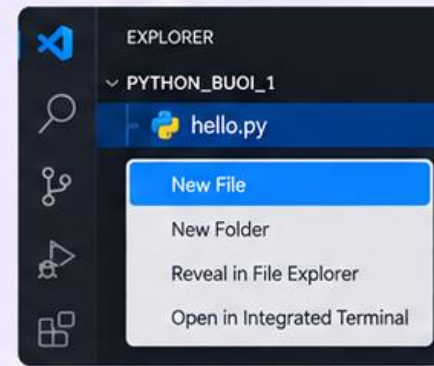
3 Tạo thư mục học tập

Tạo thư mục, ví dụ Python_Buoi_1 để lưu bài học.



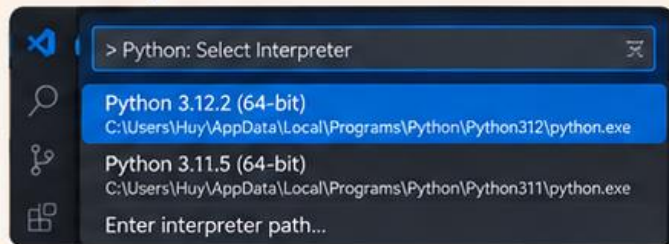
4 Tạo file hello.py

Trong thư mục, tạo file hello.py.



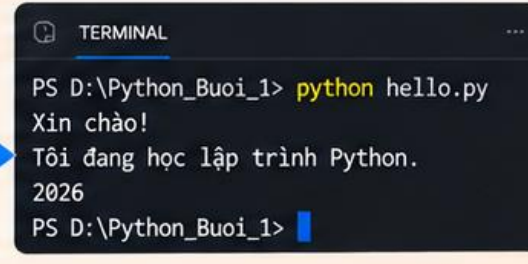
5 Chọn Python Interpreter

Chọn Python Interpreter phù hợp trong VS Code.



6 Nhập mã nguồn và chạy chương trình

Nhập mã nguồn vào file hello.py và chạy chương trình (bấm nút ▶ Run hoặc phím F5).



</> Ví dụ mã nguồn

```
1 print("Xin chào!")
2 print("Tôi đang học lập trình Python.")
3 print(2026)
```

>_ Kết quả

```
Xin chào!
Tôi đang học lập trình Python.
2026
```



Tips:

- ✓ Sử dụng phím tắt (F5) để chạy nhanh.
- ✓ Có thể tùy chỉnh theme, font chữ cho dễ nhìn.
- ✓ Giữ thư mục dự án gọn gàng, có cấu trúc.

IDE tốt
Giúp bạn đi xa hơn
trong hành trình
lập trình!



Python – Bước nhỏ hôm nay, kỹ năng lớn ngày mai!

2.3

Chạy chương trình qua Terminal

Same Code
Different Ways
Big Possibilities!



1

Cấu trúc thư mục

Giả sử file được lưu tại:

```
Python_Buoi_1/  
├──  
└── hello.py
```

2

Mở Terminal

Mở Terminal tại thư mục chứa file.

```
Terminal  
PS D:\> cd D:\Python_Buoi_1  
PS D:\Python_Buoi_1>
```

3

Chạy chương trình

Nhập lệnh và nhấn Enter:



Trên Windows (hoặc một số hệ điều hành):

```
python hello.py
```



Hoặc trên một số hệ điều hành khác:

```
python3 hello.py
```

Chỉ vài dòng lệnh
là chương trình
đã chạy được
rồi!

4

Kết quả

Chương trình sẽ hiển thị kết quả:

```
Terminal  
PS D:\Python_Buoi_1> python hello.py  
Xin chào!  
Tôi đang học lập trình Python.  
2026  
PS D:\Python_Buoi_1>
```



Phân biệt

Thành phần	Ý nghĩa
python	Gọi trình thông dịch Python.
hello.py	Tên file chứa mã nguồn.
python hello.py	Yêu cầu Python thực thi file đó.



Lưu ý

Trên một số hệ điều hành (như macOS, Linux), có thể cần dùng **python3** thay cho **python**.



Thực hành hôm nay, tự tin lập trình ngày mai!



Eat
Code
Learn
Repeat ☺

Python
Practice
Improve
Succeed



PYTHON FOR A BRIGHTER TOMORROW

2.4

Bài tập thực hành 1 – Chương trình giới thiệu bản thân

Practice
Makes Progress!



Yêu cầu

Viết chương trình hiển thị thông tin cá nhân gồm họ tên, tuổi, nghề nghiệp và mục tiêu học Python.



Gợi ý suy nghĩ

- ✓ Thông tin nào cần hiển thị?
- ✓ Sử dụng lệnh nào để in ra màn hình?
- ✓ Mỗi thông tin nên hiển thị trên một dòng hay cùng một dòng?
- ✓ Có thể làm cho kết quả hiển thị đẹp hơn không?

About
Me
In Code!



Hướng dẫn thực hiện

- 1 Tạo một file Python (ví dụ: introduce.py).
- 2 Viết chương trình theo yêu cầu.
- 3 Chạy chương trình và kiểm tra kết quả.
- 4 Thử thay đổi thông tin và chạy lại để xem kết quả.



Tiêu chí đánh giá

- ✓ Hiển thị đầy đủ 4 thông tin: họ tên, tuổi, nghề nghiệp và mục tiêu học Python.
- ✓ Chương trình chạy đúng, không lỗi.
- ✓ Kết quả hiển thị rõ ràng, dễ đọc.
- ✓ Có sự sáng tạo (nếu có).



Small Steps, Big Programmers!



2.4

Bài tập thực hành 1 – Chương trình giới thiệu bản thân

Practice
Builds Confidence!

Mã nguồn tham khảo



Mã nguồn (hello.py)

```
1 print("==== GIỚI THIỆU BẢN THÂN ====")
2 print("Họ tên: Nguyễn Văn A")
3 print("Tuổi: 20")
4 print("Nghề nghiệp: Sinh viên")
5 print("Mục tiêu: Trở thành lập trình viên Python")
6
```



Kết quả khi chạy chương trình

```
D:\Python_Buoi_1> python hello.py
==== GIỚI THIỆU BẢN THÂN ====
Họ tên: Nguyễn Văn A
Tuổi: 20
Nghề nghiệp: Sinh viên
Mục tiêu: Trở thành lập trình viên Python
```

Hãy biến
thông tin của bạn
thành chương trình
đầu tiên!



Yêu cầu mở rộng

1

Thay đổi thông tin mẫu thành thông tin của học viên.

2

Hiển thị thêm sở thích.

3

Chạy chương trình bằng cả IDE và Terminal.



Gợi ý thêm



Bạn có thể sáng tạo cách hiển thị (decorate) để chương trình đẹp hơn.



Thử sử dụng thêm các dòng trống, ký tự đặc biệt (như ==, ---).



Sau khi hoàn thành, hãy chạy thử và kiểm tra kết quả.



Python – Khởi đầu cho những điều lớn lao!



Good Ideas
Start With
Code ☺

Code
Learn
Grow
Repeat ☺

Python

Practice

Improve

Succeed

3

Biến, kiểu dữ liệu và quy tắc đặt tên

Variables
Make Ideas Real!



3.1 Khái niệm biến



Biến là tên được sử dụng để **tham chiếu đến một giá trị** trong chương trình.

Ví dụ

```
1 name = "An"      # chuỗi (string)
2 age = 20         # số nguyên (integer)
3 score = 8.5      # số thực (float)
```

Mỗi biến giống như một chiếc hộp, chứa một giá trị!

Có thể hình dung



name
(chuỗi – string)



age
(số nguyên – integer)



score
(số thực – float)

Tên biến được dùng để truy cập giá trị tương ứng.

Phép gán trong Python

Trong Python, phép gán sử dụng dấu `=`

```
1 x = 10      # gán giá trị 10 cho biến x
2 x = 25      # gán giá trị 25 cho biến x (ghi đè)
3 print(x)    # hiển thị giá trị của x
```



Kết quả

Output
25

Giải thích

Lệnh gán thứ hai làm cho tên `x` tham chiếu đến giá trị **25** thay vì **10**.



Học Python hôm nay, kiến tạo tương lai!





3.2 Các kiểu dữ liệu cơ bản

a Kiểu số nguyên – int



Dùng để biểu diễn **số nguyên**, không có phần thập phân.

**int**

(integer – số nguyên)

Số nguyên
có thể dương,
âm hoặc bằng 0.

**Ví dụ**

```
1 age = 20           # số nguyên dương
2 quantity = 5       # số nguyên dương
3 temperature = -3   # số nguyên âm
4
5 print(type(age))    # kiểm tra kiểu dữ liệu
```

**Kết quả**

```
<class 'int'>
```

**Đặc điểm của kiểu int**

- ✓ Là số nguyên (không có phần thập phân).
- ✓ Có thể là số dương, số âm hoặc 0.
- ✓ Thường dùng để đếm, tính số lượng, lưu tuổi, năm, nhiệt độ (dạng nguyên), ...

Số nguyên
rất hữu ích trong
lập trình!



Code
Learn
Grow
Repeat :)

Good
Code
Good
Life :)

Python

Practice

Improve

Succeed



Mỗi kiểu dữ liệu – một sức mạnh riêng!



3.2 Các kiểu dữ liệu cơ bản

b Kiểu số thực – float



Dùng để biểu diễn **số có phần thập phân**.

**float**

(floating point number – số thực)

Số thực có thể là số dương, âm hoặc bằng 0 và có phần thập phân.

**Ví dụ**

```
1 price = 19.5      # giá sản phẩm
2 height = 1.72     # chiều cao
3 pi = 3.14159      # giá trị π (gần đúng)
4
5 print(type(price)) # kiểm tra kiểu dữ liệu
```

**Kết quả**

```
<class 'float'>
```

**Lưu ý**

Số thực trong máy tính có thể có **sai số biểu diễn**.

```
1 print(0.1 + 0.2)
```

Kết quả thường gặp:

```
0.30000000000000004
```

Đây là đặc điểm của cách biểu diễn số thực dấu phẩy động, **không phải lỗi cú pháp Python**.

Số thực giúp chúng ta làm việc với các giá trị có phần thập phân trong thực tế!



Mỗi kiểu dữ liệu – mở ra nhiều cơ hội!



3.2 Các kiểu dữ liệu cơ bản

C

Kiểu chuỗi – str



Dùng để lưu trữ **văn bản** hoặc **chuỗi ký tự**.



str
(string – chuỗi ký tự)



Mỗi dòng code là một bước tiến!



Ví dụ

```
1 name = "Nguyen Van A" # chuỗi chứa họ tên
2 message = 'Hello Python' # chuỗi chứa thông điệp
3
4 print(name) # hiển thị giá trị của name
5 print(message) # hiển thị giá trị của message
```

“

Có thể dùng nháy đơn hoặc nháy kép

```
1 text1 = "Python" # dùng nháy kép
2 text2 = 'Python' # dùng nháy đơn
3 print(text1 == text2) # so sánh hai chuỗi
```



Kết quả

```
Output
1 True
```



Chuỗi có thể được đặt trong dấu nháy đơn (') hoặc nháy kép ("). Hai cách viết này là tương đương nhau.

Văn bản cũng
là dữ liệu!
Hãy lưu trữ nó
bằng chuỗi (str) nhé!





3.2 Các kiểu dữ liệu cơ bản

d Kiểu logic – bool



Kiểu logic (**bool**) chỉ có hai giá trị:

- **True**: đúng.
- **False**: sai.



Đúng
(True)



Sai
(False)



Ví dụ

```
1 is_student = True      # đang là sinh viên
2 is_married = False     # chưa kết hôn
3
4 print(type(is_student)) # kiểm tra kiểu dữ liệu
5
```



Kết quả

```
<class 'bool'>
```



Ứng dụng

Kiểu dữ liệu bool thường được sử dụng trong các **biểu thức điều kiện** và **cấu trúc rẽ nhánh (if, else)** để đưa ra quyết định trong chương trình.

```
if is_student:
    print("Bạn là sinh viên")
else:
    print("Bạn không phải sinh viên")
```

True hay False?
Hãy để Python
giúp bạn
quyết định!





3.2 Các kiểu dữ liệu cơ bản

e

Kiểu None



None biểu thị giá trị chưa có hoặc không có giá trị.



Chưa có dữ liệu
nhưng vẫn là
một giá trị
hợp lệ trong
Python!

None
(không có giá trị)



Ví dụ

```
1 result = None           # gán giá trị None
2 print(result)           # in ra giá trị
3 print(type(result))     # kiểm tra kiểu dữ liệu
```



Kết quả

```
Output
1 None
2 <class 'NoneType'>
```



Lưu ý

None không phải là **số 0**, **chuỗi rỗng** hay **False**, mặc dù trong một số ngữ cảnh các giá trị này đều có thể biểu thị trạng thái không có dữ liệu.

None



Chưa có giá trị

0



Số 0

""



Chuỗi rỗng

False



Giá trị logic



None nghĩa là
"chưa có giá trị".
Đừng nhầm với 0,
"" hay False nhé!



Mỗi giá trị đều có ý nghĩa riêng!



3.3. Kiểm tra kiểu dữ liệu với type()



Hàm **type()** dùng để kiểm tra kiểu dữ liệu của một biến hoặc giá trị trong Python.



int
(số nguyên)



float
(số thực)



str
(chuỗi ký tự)



bool
(logic)



NoneType
(không có giá trị)



Ví dụ

```
1 a = 100          # số nguyên
2 b = 3.14         # số thực
3 c = "Python"     # chuỗi ký tự
4 d = True         # giá trị logic
5 e = None         # không có giá trị
6
7 print(type(a))
8 print(type(b))
9 print(type(c))
10 print(type(d))
11 print(type(e))
```



Kết quả

```
1 <class 'int'>
2 <class 'float'>
3 <class 'str'>
4 <class 'bool'>
5 <class 'NoneType'>
```

Mỗi dữ liệu
đều có kiểu riêng!
Hãy dùng type()
để kiểm tra nhé!



Kiểm tra đúng kiểu – lập trình dễ hiểu hơn!



3.4. Quy tắc đặt tên biến theo PEP 8



PEP 8 là tài liệu hướng dẫn quy ước viết mã Python.



Một số quy tắc đặt tên biến:

- 1 Tên biến nên có ý nghĩa, thể hiện nội dung dữ liệu.
- 2 Sử dụng chữ thường và dấu gạch dưới để phân tách từ (snake_case).
- 3 Không bắt đầu bằng chữ số.
- 4 Không chứa dấu cách.
- 5 Không sử dụng từ khóa dành riêng của Python làm tên biến.
- 6 Tên biến phân biệt chữ hoa và chữ thường.

Tên biến tốt giúp code dễ đọc, dễ hiểu và dễ bảo trì hơn!



Quy tắc hôm nay – code đẹp ngày mai!



Tên biến hợp lệ

```
1 student_name = "An"
2 age = 20
3 total_price = 150000
4 score1 = 8.5
```

- ✓ Tên có ý nghĩa
- ✓ Sử dụng snake_case
- ✓ Không bắt đầu bằng số
- ✓ Không chứa dấu cách
- ✓ Không trùng từ khóa
- ✓ Hợp lệ theo PEP 8



Tên biến không hợp lệ

```
1 1name = "An" # Bắt đầu bằng số
2 student name = "" # Có dấu cách
3 class = "Python" # Từ khóa dành riêng
```

- ✗ 1name: bắt đầu bằng số.
- ✗ student name: chứa dấu cách.
- ✗ class: là từ khóa dành riêng của Python (keyword).

Đặt tên đúng chuẩn
Viết code chuyên nghiệp!



Ví dụ: Phân biệt chữ hoa và chữ thường

```
1 age = 20
2 Age = 25
3 print(age)
4 print(Age)
```

Kết quả

```
1 20
2 25
```



Python phân biệt chữ hoa và chữ thường.
age và Age là hai biến khác nhau.



Code
Learn
Practice
Grow :)

Python
Practice
Improve
Succeed



PEP 8 là gì?

PEP 8 là tập hợp các hướng dẫn và quy ước chính thức về phong cách viết mã (code style) cho ngôn ngữ lập trình **Python**.



Tên viết tắt **PEP 8** xuất phát từ **Python Enhancement Proposal số 8**, do **Guido van Rossum** (cha đẻ của Python) và các cộng sự công bố.



Mục đích của PEP 8

1

Tăng tính dễ đọc

Giúp mã nguồn rõ ràng, sạch sẽ và dễ hiểu cho cả người viết lẫn người đọc khác.

2

Đảm bảo tính nhất quán

Tạo ra một tiêu chuẩn chung trong cộng đồng Python, giúp việc làm việc nhóm hoặc bảo trì mã trở nên thuận lợi hơn.

Code đẹp
Học vui
Làm xa hơn



Các nội dung chính trong PEP 8

1

Quy ước đặt tên (Naming Conventions)

- Tên biến, hàm và module:** dùng kiểu **snake_case** (chữ thường, ngăn cách bằng dấu gạch dưới). Ví dụ: `my_variable`
- Tên lớp (Class):** dùng kiểu **CapWords** (viết hoa chữ cái đầu mỗi từ). Ví dụ: `MyClass`
- Hằng số (Constants):** viết hoa toàn bộ và dùng dấu gạch dưới. Ví dụ: `MAX_SIZE`

```
# Ví dụ
my_variable = 10

def calculate_sum():
    pass

class MyClass:
    pass

MAX_SIZE = 100
```

2

Định dạng khoảng trắng (Indentation & Whitespace)

- Sử dụng 4 khoảng trắng (spaces) cho mỗi cấp độ thụt lề (không dùng tab).
- Giới hạn độ dài dòng mã tối đa là 79 ký tự.
- Dùng khoảng trắng xung quanh các toán tử và sau dấu phẩy.

```
# Ví dụ
total = a + b # có khoảng trắng
if total > 10:
    print("Lớn hơn 10") # thụt lề 4 spaces
```

3

Viết tài liệu (Comments & Docstrings)

- Giải thích ngắn gọn, rõ ràng và cập nhật tài liệu khi mã thay đổi.

```
# Đây là một hàm tính tổng
def add(a, b):
    """
    Hàm tính tổng hai số.

    Tham số:
    | a (int): Số thứ nhất
    | b (int): Số thứ hai

    Trả về:
    | int: Tổng của a và b
    """
    return a + b
```





3.5. Bài tập thực hành 2 – Khai báo thông tin học viên



Yêu cầu

Khai báo các biến sau:

1



Họ tên: chuỗi ký tự.

2



Tuổi: số nguyên.

3



Điểm trung bình: số thực.

4



Đang học Python: kiểu logic.

5



Ghi chú: None.



Sau đó in giá trị và kiểu dữ liệu của từng biến.

Tự tay viết code
để khai báo và kiểm tra
thông tin của một
học viên nhé!

Small
Steps
Big
Progress



Thực hành hôm nay – vững vàng ngày mai!



3.5. Bài tập thực hành 2 – Khai báo thông tin học viên



Yêu cầu

Khai báo các biến sau:

- 1 **Họ tên:** chuỗi ký tự.
 - 2 **Tuổi:** số nguyên.
 - 3 **Điểm trung bình:** số thực.
 - 4 **Đang học Python:** kiểu logic.
 - 5 **Ghi chú:** None.
- > Sau đó in **giá trị và kiểu dữ liệu** của từng biến.



Code hôm nay
Kiến thức ngày mai
Tương lai rộng mở!



Học Python – Mở ra nhiều cơ hội!



Mã nguồn tham khảo

```
1 student_name = "Nguyen Van A" # Họ tên
2 age = 20 # Tuổi
3 average_score = 8.5 # Điểm trung bình
4 is_learning_python = True # Đang học Python
5 note = None # Ghi chú
6
7 print(student_name, type(student_name))
8 print(age, type(age))
9 print(average_score, type(average_score))
10 print(is_learning_python, type(is_learning_python))
11 print(note, type(note))
```



Lưu ý

- Bạn có thể thay đổi giá trị các biến theo thông tin của mình.
- Hãy chạy chương trình và quan sát kết quả hiển thị.



Câu hỏi củng cố

- 1 Biến `age = 20` có kiểu dữ liệu gì?
- 2 "20" và 20 có giống nhau không?
- 3 None có phải là số 0 không?
- 4 Tại sao nên đặt tên biến có ý nghĩa?

Đặt câu hỏi
Khám phá
Tiến bộ mỗi ngày!



4

Các nhóm toán tử trong Python

4.1. Toán tử số học



Dùng để thực hiện các phép tính số học.

Toán tử	Ý nghĩa	Ví dụ	Kết quả
+	Cộng	$10 + 3$	13
-	Trừ	$10 - 3$	7
*	Nhân	$10 * 3$	30
/	Chia	$10 / 3$	3.333...
//	Chia lấy phần nguyên	$10 // 3$	3
%	Chia lấy dư	$10 \% 3$	1
**	Lũy thừa	$2 ** 3$	8



Toán tử hôm nay – Giải quyết vấn đề ngày mai!

Operators
Make Code Powerful!



Cùng một
toán tử nhỏ,
tạo nên
những khả năng lớn!



Tính toán
thật đơn giản
với Python!





4.1. Toán tử số học – Ví dụ minh họa

Ví dụ:

```
1 a = 17
2 b = 5
3 print(a + b)
4 print(a - b)
5 print(a * b)
6 print(a / b)
7 print(a // b)
8 print(a % b)
9 print(a ** b)
```

Kết quả:

```
1 22
2 12
3 85
4 3.4
5 3
6 2
7 1419857
```



Giải thích kết quả:

- $a + b \rightarrow 22$
- $a - b \rightarrow 12$
- $a * b \rightarrow 85$
- $a / b \rightarrow 3.4$
- $a // b \rightarrow 3$ (chia lấy phần nguyên)
- $a \% b \rightarrow 2$ (chia lấy dư)
- $a ** b \rightarrow 1419857$ (17 mũ 5)

Cùng chạy thử
và quan sát kết quả
nhé!



Lưu ý:

// là phép chia lấy phần nguyên theo hướng làm tròn xuống, không đơn thuần là cắt bỏ phần thập phân trong mọi trường hợp.

Ví dụ:

```
1 print(-7 // 2)
```

Kết quả:

```
1 -4
```





4.2. Toán tử gán



Toán tử gán dùng để gán giá trị hoặc cập nhật giá trị của biến.

Toán tử	Ví dụ	Tương đương
=	<code>x = 10</code>	Gán 10 cho x
+=	<code>x += 5</code>	<code>x = x + 5</code>
-=	<code>x -= 5</code>	<code>x = x - 5</code>
*=	<code>x *= 2</code>	<code>x = x * 2</code>
/=	<code>x /= 2</code>	<code>x = x / 2</code>
//=	<code>x //= 2</code>	<code>x = x // 2</code>
%=	<code>x %= 3</code>	<code>x = x % 3</code>
**=	<code>x **= 2</code>	<code>x = x ** 2</code>

Gán giá trị
hôm nay,
tạo nên
kết quả lớn
ngày mai!



Code rõ ràng
Tư duy mạch lạc
Bước tiến vững chắc!



Compare
Think Clearly
Code Better!



4.3. Toán tử so sánh



Toán tử so sánh trả về kết quả kiểu **bool**, tức là **True** hoặc **False**

Toán tử	Ý nghĩa	Ví dụ	Kết quả
==	Bằng	5 == 5	True
!=	Khác	5 != 3	True
>	Lớn hơn	10 > 3	True
<	Nhỏ hơn	2 < 8	True
>=	Lớn hơn hoặc bằng	5 >= 5	True
<=	Nhỏ hơn hoặc bằng	3 <= 1	False

So sánh
là bước đầu tiên
trong việc ra
quyết định
trong chương trình!



True hay False?
Hãy để Python
giúp bạn!



So sánh đúng – Lập trình thông minh hơn!





4.4. Toán tử logic



Dùng để kết hợp hoặc phủ định các biểu thức điều kiện.

Toán tử	Ý nghĩa
and	Đúng khi cả hai điều kiện đều đúng
or	Đúng khi ít nhất một điều kiện đúng
not	Đảo ngược giá trị logic



Ví dụ:

```
1 age = 20
2 has_card = True
3
4 print(age >= 18 and has_card)
5 print(age < 18 or has_card)
6 print(not has_card)
```



Kết quả:

```
1 True
2 True
3 False
```



Bảng chân trị:

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False



Lưu ý:

- Các toán tử logic thường được sử dụng trong câu lệnh điều kiện (**if**) để kiểm tra nhiều điều kiện cùng lúc.
- Bạn có thể kết hợp nhiều toán tử logic trong cùng một biểu thức.
- not** có độ ưu tiên cao hơn **and** và **or**.

Tư duy logic tốt
là chìa khóa
để viết những
chương trình hiệu quả!



Good
Code
Good
Life :)





4.5. Toán tử membership



Dùng để kiểm tra một phần tử có nằm trong một chuỗi hoặc tập hợp dữ liệu hay không.

Toán tử	Ý nghĩa
<code>in</code>	Có nằm trong
<code>not in</code>	Không nằm trong



Ví dụ 1: Sử dụng với chuỗi

```
1 language = "Python"
2 print("Py" in language)
3 print("Java" in language)
4 print("Java" not in language)
```



Kết quả:

```
1 True
2 False
3 True
```



Ví dụ 2: Sử dụng với danh sách

```
1 fruits = ["apple", "banana", "orange"]
2 print("banana" in fruits)
3 print("grape" not in fruits)
```



Kết quả:

```
1 True
2 True
```

Kiểm tra
sự tồn tại
dễ dàng hơn
với toán tử
membership!



Python giúp bạn
kiểm tra dữ liệu
nhẹ nhàng!



Kiến thức nhỏ – Ứng dụng lớn!



4.6. Bài tập nhanh trên lớp



Cho đoạn chương trình sau:



Python Code

```
1 a = 15
2 b = 4
3
4 print(a // b)
5 print(a % b)
6 print(a > b and b != 0)
  print(5 in [1, 3, 5, 7])
```



Yêu cầu:

Học viên dự đoán kết quả trước khi chạy chương trình.



Gợi ý thảo luận:

- Mỗi câu lệnh sẽ in ra giá trị gì?
- Vì sao?
- Hãy ghi kết quả dự đoán của bạn vào giấy hoặc thảo luận nhóm.

Dự đoán trước khi chạy
– Học chủ động hơn!
😊



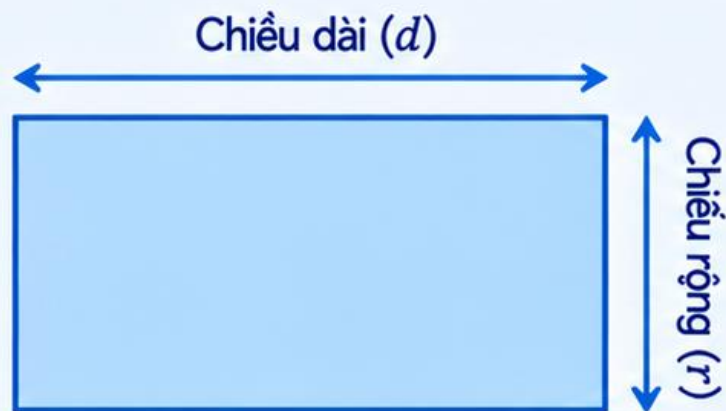


Bài tập 1. Tính chu vi và diện tích hình chữ nhật



Yêu cầu

Nhập chiều dài và chiều rộng của hình chữ nhật, tính chu vi và diện tích.



Công thức

Chu vi:

$$P = 2 \times (d + r)$$

Diện tích:

$$S = d \times r$$

Thử sức
vận dụng kiến thức
để giải quyết
bài toán thực tế nhé!





Bài tập 2. Tính hóa đơn mua hàng



Yêu cầu

Một khách hàng mua 3 sản phẩm, mỗi sản phẩm có giá 150.000 đồng.

Tính tổng tiền, số tiền giảm giá 10% và số tiền phải thanh toán.



Thông tin mua hàng

STT	Sản phẩm	Đơn giá
1	Sản phẩm 1	150.000 đ
2	Sản phẩm 2	150.000 đ
3	Sản phẩm 3	150.000 đ



Cần tính

- 1 Tổng tiền (trước khi giảm giá)
- 2 Số tiền giảm giá (10%)
- 3 Số tiền phải thanh toán (sau khi giảm giá)

Áp dụng kiến thức đã học để giải bài toán thực tế! Bạn làm được mà!





Bài tập 3. Tính điểm trung bình



Yêu cầu

Cho điểm Toán, Văn, Anh lần lượt là 8, 7.5 và 9.

Tính điểm trung bình và kiểm tra xem điểm trung bình có lớn hơn hoặc bằng 8 hay không.



Thông tin điểm số

Môn học	Điểm
Toán	8
Văn	7.5
Anh	9

Tư duy logic
Giải quyết vấn đề
Từng bước một!



Cần thực hiện

1

Tính điểm trung bình của 3 môn.

2

Kiểm tra xem điểm trung bình có lớn hơn hoặc bằng 8 hay không.



Lập trình hôm nay – Cơ hội ngày mai!





Bài tập 4. Bài tập tự luyện



Yêu cầu

- 1 Khai báo hai biến $a = 12$ và $b = 5$.
In ra tổng, hiệu, tích, thương và số dư.

Tự luyện
để thành thạo
hơn nhé!



Gợi ý

- Sử dụng các toán tử số học: $+$, $-$, $*$, $/$, $\%$
- Sử dụng hàm `print()` để in kết quả.
- Có thể in kèm nhãn (ví dụ: "Tổng =", ...) để kết quả rõ ràng hơn.



Kỹ năng ôn luyện

- ✓ Khai báo biến
- ✓ Toán tử số học
- ✓ Toán tử chia lấy phần dư (%)
- ✓ Xuất kết quả ra màn hình





Câu hỏi kiểm tra cuối buổi

?

Câu 1.

Kiểu dữ liệu của giá trị 3.5 là gì?

A

int

B

float

C

str

D

bool

3.5



Think
Code
Be Creative!



Ôn lại kiến thức
để tiến xa hơn
mỗi ngày!





Câu hỏi kiểm tra cuối buổi

?

Câu 2.

Toán tử nào dùng để so sánh hai giá trị bằng nhau?

A

=

B

!=

C

==

D

+=



Think
Code
Be Creative!



Ôn lại kiến thức
để tiến xa hơn
mỗi ngày!





Câu hỏi kiểm tra cuối buổi

?

Câu 3.

Kết quả của $17 \% 5$ là gì?

A

2

B

3

C

3.4

D

5

 $17 \% 5$

Phép chia
lấy số dư

Think
Code
Be Creative!



Ôn lại kiến thức
để tiến xa hơn
mỗi ngày!



Kiến thức hôm nay – Cơ hội ngày mai!



Câu hỏi kiểm tra cuối buổi

?

Câu 4.

Kết quả của True and False là gì?

A

True

B

False

C

None

D

1



Think
Code
Be Creative!



Ôn lại kiến thức
để tiến xa hơn
mỗi ngày!





Câu hỏi kiểm tra cuối buổi

?

Câu 5.

Biểu thức 'Py' in 'Python' trả về gì?

A

False

B

None

C

True

D

Lỗi



Think
Code
Be Creative!



Ôn lại kiến thức
để tiến xa hơn
mỗi ngày!





Bài tập về nhà

Think
Code
Be Creative!



1

Bài 1 – Tính tiền điện



Khai báo số điện tiêu thụ và đơn giá điện. Tính tổng tiền điện theo công thức đơn giản:

Tiền điện = Số điện × Đơn giá

2

Bài 2 – Đổi đơn vị



Nhập hoặc khai báo số phút, chuyển đổi thành số giờ và số phút còn lại bằng toán tử `//` và `%`.

Ví dụ: 130 phút → 2 giờ 10 phút

3

Bài 3 – Tính lương



Khai báo lương cơ bản, số ngày công và phụ cấp. Tính tổng lương theo công thức do giảng viên cung cấp.

(Sử dụng biến, toán tử và biểu thức)

4

Bài 4 – Thử thách



Viết chương trình tính tổng số tiền mua hàng của 3 mặt hàng, tính phần trăm giảm giá và in hóa đơn với định dạng dễ đọc.

Gợi ý: Sử dụng biến, toán tử, format chuỗi (`f-string`) để trình bày kết quả.

Luyện tập
mỗi ngày
để giỏi hơn!



Kiến thức hôm nay – Cơ hội ngày mai!